

## - Hugo Habercore:

- The disregard for cleanliness and personal hygiene standards in the kitchen is a significant contributor to foodborne illness outbreaks in dining establishments. **This typically results from a lack of regular monitoring**, as these criteria are not followed **until the health inspector is present (Mostly in local eateries)**.
  - The goal of this project is to **monitor restaurant kitchen cleanliness and provide daily updates about each chef regarding their compliance with hygienic protocols**. These reports will be especially crucial when the area's health inspector visits, giving them a more **accurate and lucid perspective**. It is also important for the chef and the restaurant owner himself to notice mistakes and try not to repeat them.
- The project is an **automated monitoring system for chefs in restaurants**, powered by **Computer Vision and AI**.  
It works in **real-time** through a webcam or an external camera connected to the desktop, and evaluates the chef's compliance with hygiene and safety standards.

## ○ What does health inspector look for?

### Personal hygiene

Your team's good personal hygiene is critical for reducing the spread of pathogens and maintaining a safe restaurant, and inspectors will take notice. If employees don't follow handwashing guidelines, wear dirty clothes, or show up to work sick, your operation could be at risk.

Employees should **wash their hands after:**

- Using the restroom
- Leaving and returning to kitchen/prep areas
- Touching their face, hair, or clothing
- Eating, drinking, smoking, or chewing gum

- Clearing tables, busing dirty dishes, or taking out the trash
  - Handling raw meat, poultry, or seafood
  - Handling chemicals
- Employees wear disposable gloves and change them frequently.
  - Employees don't use tobacco products anywhere in the establishment.

### **Cleaning/Sanitation**

- All surfaces are **smooth, easily cleanable**, and frequently washed.
- All **equipment is kept clean** and in proper working order and meets ANAB standards.
- **Chemicals and cleaning supplies are kept away** from food surfaces when in use.
- Cleaners are completely washed away from food surfaces after cleaning.
- Chemicals are properly labeled and stored in a separate area from food storage.
- Sanitizers are tested using approved chemical test strips.

### **Pest Control**

- Dumpsters are on a level surface with good drainage at an adequate distance from the establishment. They are frequently cleaned, kept tidy, and covered.
- Trash is disposed of regularly, there is no overflow of trash, and trash receptacles are covered.
- There are no signs of cockroach or rat droppings or urine, egg casings, or fly infestation.
- Doors are self-closing and are kept closed to avoid allowing pests in the facility.

More info and details about cleanliness and special hygiene :

<https://www.foodstandards.gov.au/business/food-safety/health-and-hygiene-advice-for-food-handlers>

<https://www.health.vic.gov.au/food-safety/personal-hygiene-for-food-handlers>

- If team members display signs of illness, send them home until they've fully recovered.

## *Define Key Features*

### 1- Chef identification (Face Recolonization):

- The system recognizes each chef by face.
- Generates **individual reports** for each chef.

### 2- Chef Uniform Detection:

- Chef jacket.
- Apron.
- Cap/Hat (Head Nets)
- Gloves.
- Face Mask.



## A. Personal Hygiene

- Detect if the chef:
  - Is wearing **clean clothes**.
  - Has **hairnet or cap**.
  - Is using **face mask**.
  - Is wearing **gloves**.
  - **Handwashing activity** (optional advanced feature).
- Detect if the chef **touches face, hair, or clothes** during work.

## B. Cleaning/Sanitation

- Detect if **work surfaces** are clean or cluttered.
- Detect **sanitizer presence** (optional advanced).

## C. Reporting System

- Generate **daily reports**:
  - For **each chef** (compliance percentage).
  - Send to **owner / health inspector**.
  - Export as PDF or Excel.
- Possibly add **real-time alerts** (e.g., "Chef not wearing gloves").

## D. Real-Time Alerts:

- Audio or text alert on the screen if violation is detected.
- Example: "Please wear a mask" or "Do not touch food with bare hands".

## E. Database + Dashboard:

- Store all reports in a database.
- Provide a Dashboard with statistics (overall compliance rate, number of violations, best/worst performing chefs).

# What Data Do You Need?

## 1. Chef Uniform Dataset

- What to include:
    - Images of chefs wearing a full uniform (jacket, apron, hat, gloves, mask).
    - Images of chefs missing specific items (e.g., no hat, no gloves, no mask).
  - Purpose: Detect missing protective gear as a violation.
  - Where to get it:
    - Use existing datasets like [Open Images Dataset](#) or search for "chef uniform dataset" on [Kaggle](#) and [Roboflow Universe](#).
    - Use image augmentation (rotation, brightness, blur) to increase dataset size.
- 

## 2. PPE Detection Data (Hat, Mask, Gloves, Apron)

- What to include:
  - Pre-labeled images for detecting masks, gloves, and hairnets.
- Purpose: Identify PPE compliance in real-time.
- Where to get it:
  - Mask detection datasets: [Face Mask Detection Dataset on Kaggle](#).
  - Gloves & PPE: Use datasets from Roboflow PPE Detection Projects.

- For apron/chef coats: You may need to create a custom dataset using your kitchen photos and label them using Labelling or Roboflow.
- 

### 3. Face Recognition Data (For Chef Identification)

- What to include:
    - 20–50 images per chef from different angles and lighting conditions.
  - Purpose: Link detected violations to the correct chef.
  - Where to get it:
    - Collect internally (with consent) using your restaurant's cameras.
    - Store images locally for privacy.
- 

### 4. Behavior Recognition Data

- What to include:
  - Washing hands (in sink area, using soap).
  - Smoking in kitchen (strict violation).
  - Touching face/hair (violation).
  - Touching food without gloves.
  - Cooking properly (correct handling of food).
  - Standing idle / leaving work for too long.

- Washing hands after required situations (after touching raw meat, after break).
  - Frequent glove changing (detect if gloves remain unchanged for long time).
  - Where to get it:
    - Public datasets:
      - [Kaggle Action Recognition Datasets](#) (search for "action recognition" or "kitchen activities").
      - [UCF101 Action Recognition Dataset](#) (general actions, but can fine-tune).
      - [Epic Kitchens Dataset](#) (kitchen-related actions).
    - Custom Data Collection:
      - Record short video clips in your kitchen for specific scenarios (e.g., washing hands, smoking).
      - Use frame extraction to convert videos to labeled images for training.
    - Augment videos/images for diversity.
- 

#### ✓ Extra Notes on Data Acquisition

- Use Roboflow or CVAT for labeling and augmenting custom datasets.
- For glove-change tracking, you will combine object detection (glove) + time-based analysis to check how long gloves were worn.
- Smoking detection can use object detection for cigarettes or pose/action recognition for smoking gestures.

- Washing-hand-required situations need contextual recognition (e.g., detecting raw meat handling → expecting handwashing after).

## Technical Requirements

### 1. Development Environment

- Language: Python
  - IDE: VSCode / PyCharm / Jupyter Notebook
  - Core Libraries:
    - opencv-python → live video & image processing
    - torch / torchvision or tensorflow/keras → model training & inference
    - ultralytics (YOLOv8/YOLOv11) → PPE & uniform detection
    - numpy, pandas, scikit-learn → data handling & analysis
    - matplotlib / seaborn → visualization
- 

### 2. Operating Modes

- Real-Time Mode:
  - Live monitoring using a connected camera (USB/Webcam).
  - Instant alerts and on-screen bounding boxes.
- Video Upload Mode:
  - User uploads a recorded kitchen video.



- System analyzes it frame-by-frame and generates a full compliance report.
- 

### **3. Chef Uniform & PPE Detection**

- **Model:** YOLOv8/YOLOv11 object detection
  - **Classes:** Jacket, Apron, Hat, Gloves, Mask (+ NoJacket, NoApron, NoHat, NoGloves, NoMask)
  - **Dataset Sources:** Roboflow PPE datasets, Kaggle datasets, custom chef images
- 

### **4. Face Recognition**

- **Libraries:** face\_recognition, deepface, or dlib
  - **Purpose:**
    - Identify each chef by face
    - Store individual compliance reports per chef
- 

### **5. Behavior Recognition**

- **Target Behaviors:**
    - Washing hands
    - Touching face/hair
    - Touching food without gloves
    - Cooking normally
    - Standing idle/away for too long
  - **Methods:**
    - OpenPose for hand & pose tracking
    - Action recognition models (I3D, SlowFast) for complex behaviors
- 

### **6. Reports & Database**

- **Database:** SQLite (lightweight) or MySQL (scalable)
  - **Reports:**
    - pandas + matplotlib → stats & plots
    - reportlab or fpdf → PDF reports
    - openpyxl → Excel reports
- 

## **7. Real-Time Alerts**

- **Voice Alerts:** pyttsx3 (text-to-speech)
  - **On-Screen Alerts:** OpenCV overlay (“Please wear mask”)
- 

## **8. Desktop Application (CustomTkinter)**

- **Framework:** customtkinter
  - **Main Features:**
    - **Dual Mode:** switch between Real-Time Monitoring or Video Upload Analysis
    - Live camera feed with bounding boxes & alerts
    - Sidebar navigation (Dashboard, Reports, Settings, Upload Video)
    - Alerts panel (real-time or post-analysis feedback)
    - Reports export (PDF/Excel)
    - Database integration (chef compliance history)
- 

## **9. Dashboard (inside CustomTkinter)**

- **Embedded visualizations using matplotlib**
- **Show:**
  - Compliance rate per chef
  - Violations per category (mask, gloves, hat, etc.)
  - Chef ranking (best/worst compliance)

- Daily/weekly/monthly summaries

---

#### **Additional Suggestions (Value-Added Features)**

- **Multi-Chef Tracking:** detect and track multiple chefs in the same kitchen
- **AI Feedback:** automatic suggestions in real-time (“Please wash your hands”, “Put on gloves”)
- **Activity Monitoring:** track idle time vs. working time per chef
- **Integration with Restaurant Management:** reports accessible by managers
- **Voice + Visual Alerts:** optional voice guidance alongside screen pop-ups
- **Cloud Sync (Optional):** upload reports to a central server for chain restaurants

---

#### **10. Hardware**

- **Camera:** HD webcam or external USB camera
- **System:** Laptop/PC with NVIDIA GPU (preferred)
- **Alternative:** Google Colab / Kaggle for training if local GPU unavailable

---

**Full**

**Flow:**

**Camera / Uploaded Video → YOLO (PPE Detection) + Face Recognition + Behavior Recognition → Database & Logs → CustomTkinter App (Live Mode + Video Mode + Alerts + Dashboard + Reports).**

## ✅ Full Technical Concepts You Must Master

(Organized in phases for clarity)

---

### 1. Python & Core Libraries (Must Know)

- Why: Python is the backbone of AI, CV, and GUI development.
  - What to learn:
    - Python basics: loops, functions, classes, OOP.
    - Data handling: numpy, pandas.
    - Image handling: opencv-python.
    - Visualization: matplotlib.
  - Resources:
    - [W3Schools Python](#)
    - FreeCodeCamp Python (YouTube)
    - OpenCV Crash Course: [Learn OpenCV in 4 Hours](#)
  - Time Needed: 3–4 days.
- 

### 2. Computer Vision Fundamentals

- Why: All detections rely on CV concepts.
- What to learn:
  - Image processing basics (thresholding, contours, color spaces).
  - Object detection intro (YOLO, SSD).
  - Face detection vs recognition.
- Tools: OpenCV + YOLOv8.
- Resources:
  - OpenCV Docs + Projects: [OpenCV.org](#)
  - YOLOv8 official tutorial (Ultralytics).

- Time Needed: 4 days.
- 

### 3. Deep Learning for Object Detection

- Why: PPE detection and action recognition need robust models.
  - What to learn:
    - CNN basics.
    - YOLOv8 for detection tasks.
    - Transfer Learning (fine-tuning on custom datasets).
  - Tools: PyTorch OR TensorFlow (choose one; PyTorch is easier with YOLOv8).
  - Resources:
    - YOLOv8 Docs: [Ultralytics YOLOv8](#)
    - Course: *Deep Learning for CV with PyTorch* (YouTube free).
  - Time Needed: 5–6 days.
- 

### 4. Face Recognition

- Why: Identify chefs for reports.
  - What to learn:
    - Pre-trained models like FaceNet, Dlib, or face\_recognition library.
    - Embeddings & face matching.
  - Resources:
    - [Face Recognition with Python](#)
    - Medium tutorials on FaceNet + Dlib.
  - Time Needed: 2 days.
- 

### 5. Action/Behavior Recognition

- Why: Detect washing hands, smoking, touching hair, etc.

- **What to learn:**
    - Pose estimation: OpenPose / MediaPipe.
    - Action recognition models: I3D, SlowFast.
    - Temporal analysis (video → frame sequence).
  - **Resources:**
    - MediaPipe Docs.
    - Action Recognition: [UCF101 with PyTorch](#).
  - **Time Needed:** 5–6 days.
- 

## 6. Real-Time Inference + Integration

- **Why:** Combine everything (YOLO + Face + Actions) in real-time.
  - **What to learn:**
    - OpenCV for live video streams.
    - Multi-threading for smooth performance.
    - Overlay text & bounding boxes.
  - **Resources:**
    - OpenCV Video Streaming tutorials.
  - **Time Needed:** 2 days.
- 

## 7. GUI Development (Desktop App)

- **Why:** For an interactive app with reports & dashboards.
- **What to learn:**
  - customtkinter (modern Tkinter UI).
  - Integrate OpenCV feed into Tkinter window.
- **Resources:**
  - [CustomTkinter Docs](#).

- **Time Needed: 2 days.**
- 

## **8. Reporting & Dashboard**

- **Why: Generate compliance reports for chefs.**
  - **What to learn:**
    - reportlab or fpdf for PDFs.
    - openpyxl for Excel reports.
    - Plotting compliance graphs using matplotlib.
  - **Time Needed: 1 day.**
- 

## **9. Database & Logging**

- **Why: Store historical data (compliance, violations).**
  - **What to learn:**
    - SQLite (lightweight, easy).
    - CRUD operations.
  - **Time Needed: 1 day.**
- 

## **10. ESP32-CAM (Optional)**

- **Why: If you want IoT integration later.**
  - **What to learn:**
    - Basics of ESP32 (Arduino IDE / MicroPython).
    - Streaming video from ESP32 to Python app.
  - **Resources:**
    - Random Nerd Tutorials: ESP32-CAM projects.
  - **Time Needed: 2 days.**
-

## ✅ How to Study This in 1 Month (Aggressive Plan)

### Week 1:

- Python + OpenCV basics → 4 days.
- CV Fundamentals → 3 days.

### Week 2:

- YOLOv8 + Object Detection → 4 days.
- Face Recognition → 2 days.
- Start GUI basics (CustomTkinter) → 1 day.

### Week 3:

- Behavior Recognition (OpenPose + MediaPipe + simple action recognition) → 5 days.
- Real-Time Integration (OpenCV + GUI) → 2 days.

### Week 4:

- Reports + Dashboard → 1 day.
- Database → 1 day.
- ESP32 basics (optional) → 2 days.
- Final Integration + Testing → 3–4 days.

---

🔥 Total Time = 28 days hardcore (4–6 hrs/day).  
Yes, it's doable if you don't waste time on deep theory and focus on practical coding + ready models.